

全端軟體工程師｜情境與想法交流

💬 **我們想了解的，不只是你的答案，還有你如何理解問題、衡量不同的可能，並做出選擇。**
以下是九個我們在軟體開發中常會遇到的情境，想邀請你一起想想，遇到這些情境時會怎麼處理。文末另有一個話題，歡迎分享你與 AI 合作的經驗。

怎麼回答，由你決定。 文字、圖解、程式碼、簡報或其他形式都可以。你可以逐題回答，也可以把相關想法串在一起，標明對應的情境即可。

AI、搜尋引擎與開發工具都可以自由使用。遇到陌生的概念，可以先查資料，再說說你的理解。這次以討論想法為主，不必另外完成一套應用程式。

遇到資訊不足的地方，請說明你的假設或想確認的問題。談到驗證時，也請區分實際做過的測試與打算採取的方法。

完成後，將回答或連結回傳 HR。面談時，我們會從你的內容接著聊。

以下票務規則是為了討論而設定，與實際售票政策無關；各題以題目提供的條件為準。

情境一｜一張票，誰的權益？

情境二｜換了金流服務商，哪些事不該變？

情境三｜一句需求，離開發還有多遠？

情境四｜兩個櫃台，能領兩次嗎？

情境五｜明明用了 async，為什麼還是卡？

情境六｜登入了，就能操作嗎？

情境七｜從工作分支，到一起交付

情境八｜當 AI 成為搭檔，你會怎麼開始？

情境九｜數字對不上，差異從哪裡來？

延伸分享｜有沒有一次，你改變了與 AI 合作的方式？

情境一｜一張票，誰的權益？

可以先了解 DDD（領域驅動設計）如何用模型表達業務中的角色、關係與規則，再看看下面這個情境。

甲購買一筆訂單，包含電子票 T1、T2。甲把 T1 轉贈給乙，T2 仍由甲持有。

本題規則：只有已付款、有效票券的**目前持有人**能使用該票領贈品；每張票對同一贈品種類最多領取一次，轉贈不會增加次數。贈品核銷與入場驗票是兩種不同操作。

1. 在這裡，購買人、目前持有人、訂單、票券與贈品兌換紀錄各扮演什麼角色？購買人與目前持有人可能不是同一個人，你會怎麼在資料與操作中區分這兩個角色？
2. 挑一條你認為重要的業務規則，說明你會怎麼把它放進模型。
3. 前台顯示可以領取，與後台執行核銷，應該怎麼使用這條規則？為什麼？

情境二｜換了金流服務商，哪些事不該變？

可以從 Hexagonal Architecture 的 Port、Adapter 與 Use Case 開始理解這個情境，留意「程式呼叫誰」與「原始碼依賴誰」的差別。

退票流程需要判斷資格、建立退款申請、呼叫外部金流、記錄處理結果，並寄送狀態通知。外部金流有自己的 SDK、欄位名稱與錯誤碼。

目前由會員透過網站申請退票，未來也可能讓客服從後台協助處理。團隊正在評估更換金流服務商；假設新舊服務商都支援這次需要的退款功能，退票資格與退款金額規則保持不變。先談各部分的責任與依賴，跨系統一致性的細節暫時放在一旁。

1. 判斷可否退票、協調退款流程、呼叫金流 SDK、存取資料、解析 HTTP 請求，這些工作你會怎麼分？請選一條依賴關係說明。
2. 挑一個需要外部系統協助的工作，試著設計它的 Port。你希望傳入什麼、得到什麼結果？
3. 還沒有金流測試帳號，你會先測哪些規則或流程？哪些部分要等真正串接後才能確認？

情境三｜一句需求，離開發還有多遠？

這裡的 SDD 指 Spec-Driven Development（規格驅動開發）。可以先了解規格如何幫助團隊確認需求、實作與驗收，以及需求改變後，這幾件事如何一起調整。

PM 目前只說：「會員可以在網站申請退票。」一筆訂單可能包含多張、不同場次的票券，其餘政策尚未提供。

1. 你最想先確認哪三件事？請排出順序，說說答案會影響哪些決定。
2. 先採用幾個合理假設，整理一份規格草稿：這次要做到哪裡、哪些先不做、誰可以申請、可以退哪些票、流程有哪些狀態，以及失敗或重複申請時怎麼處理。請標明假設，還不能決定的地方就留下問題。
3. 以提交退票申請為例，說明申請被受理、不符合資格，以及重複提交時，各應得到什麼結果。

情境四｜兩個櫃台，能領兩次嗎？

這一題會用到 Race Condition（競態條件）、資料庫 Transaction 與唯一性約束。可以先了解它們分別能解決什麼問題。

兩個櫃台同時對同一張票、同一種贈品送出核銷請求，分別由兩個 API 服務實例處理。它們共用同一個關聯式資料庫。規則是同一張票對同一贈品只能核銷一次，資料庫目前沒有對這個組合設定唯一性約束。

假設登入、持有人資格與票券有效性都已通過檢查。本題只討論建立核銷紀錄；不涉及點數扣款、實體發放或贈品總庫存。

以下用 `db` 示意資料庫操作：

```
async function redeem(
  ticketId: string,
  giftCodeKey: string,
) {
  const existing = await db.findRedemption(
    ticketId,
    giftCodeKey,
  );
  if (existing) return { status: 'already_redeemed' };

  await db.insertRedemption({ ticketId, giftCodeKey });
  return { status: 'redeemed' };
}
```

1. 試著排出兩個請求的執行順序，看看問題可能出在哪裡。都有 `await`，就能避免重複核銷嗎？
2. 你會怎麼做最小幅度的修正？靠什麼避免重複，另一個請求又應該收到什麼結果？

3. 你會怎麼測試兩個請求同時進來的情況？如果只是先後呼叫兩次，可能漏掉什麼？

情境五｜明明用了 async，為什麼還是卡？

可以先了解 Node.js 的 Event Loop，以及同步運算與非同步 I/O 的差別，再看下面這段程式。

一個 Node.js 程序同時處理購票 API 與通知準備工作。每次批次產生大量通知信內容時，購票 API 的回應就明顯變慢。

`renderLargeTemplateSync` 會進行耗用 CPU 的同步 JavaScript 運算，並與購票 API 在同一個執行緒中執行：

```
async function prepareMessages(orders: Order[]) {
  return orders.map(order =>
    renderLargeTemplateSync(order)
  );
}

const messages = await prepareMessages(orders);
await mailQueue.addBulk(messages);
```

1. 同事認為「函式有 `async`，外面也有 `await`，所以不會阻塞」。你同意嗎？為什麼？
2. 你會再看哪兩項數據或紀錄，判斷 API 變慢的原因？
3. 你會先改哪個地方？怎麼確認有改善？請分開考慮「準備通知」與「寄送通知」的工作。

情境六｜登入了，就能操作嗎？

這裡會談到 CORS、登入驗證、操作授權與 HTTP 方法。可以先釐清它們各自負責什麼。

你收到一份退款功能的 PR，準備進行審查。這次的業務規則是：**只有訂單的原購買人可以申請自助退款。**

這次提交的修改包含以下做法：

- 使用 `GET /refund?orderId=...`，呼叫後會直接建立退款申請。
- API 只驗證請求者已登入，依 `orderId` 取得訂單後就執行，未比對原購買人。
- CORS 僅允許官網來源，因此作者認為不需要其他授權檢查。
- 測試只有一個以 mock 模擬成功的案例；同時修改了多個不相關模組的檔名。

1. 你會讓這份 PR 通過嗎？挑出最需要先處理的兩個問題，說明可能造成什麼後果。
2. 你會補哪個測試？預期應該得到什麼結果？
3. 那些不相關的檔名修改，你會怎麼安排？怎麼讓這次變更容易審查，出了問題也方便回退？

情境七｜從工作分支，到一起交付

可以先了解工作分支、rebase，以及 PR（Pull Request）／MR（Merge Request）的用途，試著串起一次修改從開發到合併的過程。

你準備替售票系統新增「贈品核銷紀錄查詢」。團隊以 `main` 作為整合分支，每項工作先在自己的工作分支進行。

開發期間，同事已把其他修改合併進 `main`，並建議你在送交審查前，將自己的工作分支 rebase 到最新的 `main`。團隊透過 PR／MR 討論與審查修改，再決定是否合併。

另外一項任務只是整理既有核銷程式的結構，不改變原本行為。你看過 `feature/`、`feat/`、`refactor/` 等分支前綴，希望讓同事從名稱就能理解工作目的。

1. 工作分支有什麼用途？請分別替「新增查詢功能」與「整理既有程式、不改變行為」取一個分支名稱，說明前綴想傳達什麼。這些前綴是 Git 的內建功能，還是團隊約定？
2. 把自己的工作分支 rebase 到最新 `main`，是在做什麼？請用自己的方式說明前後差異，以及操作前需要留意的一件事。
3. PR 與 MR 分別指什麼？它們在協作流程中扮演什麼角色？請說明把工作分支 push 上去、建立 PR／MR、合併進 `main`，這幾件事有什麼不同。

情境八｜當 AI 成為搭檔，你會怎麼開始？

可以先了解 AI Agent 如何讀取資料、使用工具，並根據執行結果繼續工作，再想想你會怎麼和它分工。

你接到一項既有系統的小改動，準備與 AI Agent 一起完成。你可以沿用前面的票務情境，例如新增「贈品核銷紀錄查詢」，也可以選一個自己熟悉的工作。

從理解問題到準備交付，你會怎麼安排合作？可以分享實際經驗，也可以提出合作設想，說明你會這樣安排的理由。

1. 開始之前，你會先給 Agent 哪些資訊，或請它先看什麼？為什麼？
2. 哪些工作會交給它？什麼時候你會停下來確認，或調整方向？
3. Agent 說「完成了」，你會先看什麼？什麼情況下會接受結果，什麼情況下會請它再改？

情境九｜數字對不上，差異從哪裡來？

某場次結束後，營運發現後台顯示該場次售出 10,012 張票，但另一份報表顯示同場次只有 9,847 張。現在還不知道是統計口徑不同、查詢邏輯有問題，還是資料本身真的異常。

已知一筆訂單可以包含多張票；訂單可能處於待付款、已付款或取消等狀態，部分票券也可能已退款。訂單、票券與退款資料存在 PostgreSQL，場次設定則存在 MongoDB。

1. 你會先確認哪些定義，再開始查資料？哪些差異可能只是雙方「算的東西不一樣」？
2. 你會怎麼一步一步縮小問題？可以寫 SQL／Query，也可以直接說明你的查法與順序。
3. 當你找到造成差異的資料或計算方式，會怎麼判斷這是統計定義不同、查詢問題，還是資料異常？你會留下哪些資訊，讓別人也能確認你的結論？

延伸分享｜有沒有一次，你改變了與 AI 合作的方式？

這題自由分享，也可以留到面談再聊，或直接略過。

回想一次你與 AI 合作的經驗，順利或不順利都可以。當時你怎麼做？後來注意到什麼？有沒有因此改變下一次的做法？

一個小發現也很好，也可以分享其他 AI 工具的使用經驗，或還想試試看的做法。